BACKEND DEVELOPMENT PROJECT

TaskFlow API

Project introduction | Kübra Kovayçin | 19 October 2026

Purpose and scenario

A software team needs a simple way to record work, assign responsibility, and follow progress. TaskFlow API provides a REST interface for managing these tasks. A task carries a title, description, assignee, status, and priority, allowing the team to identify who is responsible and what needs attention. The project applies Node.js, Express, routing, middleware, PostgreSQL persistence, and HTTP response conventions.

Core behavior

Clients can create a task, list all tasks, retrieve one task, update selected fields, and delete a task. Each task receives a server-generated UUID and creation/update timestamps. Status values are pending, in_progress, and completed; priority values are low, medium, and high. New tasks default to pending status and medium priority.

Implementation

Express routes connect task endpoints to controllers, which delegate task operations to the service. The service shares validation rules between creation and partial updates and persists tasks in PostgreSQL using parameterized SQL. The database pool reads DATABASE_URL from the environment. A logger records incoming requests, while application configuration and server startup remain separate modules.

Task data model

Field	Type	Rule
id	string	UUID generated by the server; read-only
title	string	Required on creation; non-empty after trimming
description	string	Optional; defaults to an empty string
assignee	string	Required on creation; non-empty after trimming
status	string	pending | in_progress | completed
priority	string	low | medium | high
createdAt	string	Server-managed ISO timestamp; read-only
updatedAt	string	Server-managed ISO timestamp; read-only

API AND VERIFICATION

System summary

Base URL: http://localhost:3000

Method	Path	Purpose	Success
GET	/health	Check server availability	200
POST	/tasks	Create a task	201
GET	/tasks	List all tasks	200
GET	/tasks/:id	Retrieve one task	200
PATCH	/tasks/:id	Update selected fields	200
DELETE	/tasks/:id	Delete a task; empty response	204

Replace :id with an actual task ID returned by the API. POST and PATCH requests use JSON bodies.

Validation and error handling

Creation requires a title and assignee containing non-whitespace text. Description, status, and priority are optional, but supplied values must be valid. Requests must contain a non-empty JSON object with supported editable fields only. Clients cannot replace IDs or timestamps. PATCH preserves fields that are omitted from the request.

Invalid input receives HTTP 400. A missing task or unmatched endpoint receives HTTP 404 with a specific message. Internal failures receive HTTP 500 with a generic client message, while their details are logged on the server.

Logging and persistence

The logger records the UTC timestamp, HTTP method, and original request URL before CORS handling and JSON parsing, including preflight and malformed JSON requests. PostgreSQL stores tasks with UUIDs and timestamp columns. The setup script creates the table from database/schema.sql and preserves existing rows. Saved tasks remain after an API restart. The /health endpoint checks HTTP availability only.

Manual verification and evidence

The Postman verification sequence covers five successful operations: create, list, detail, update, and delete. A created task ID is used for detail, update, and deletion. For each operation, a screenshot should show the method, full URL, status code, and response. HTTP 204 has an empty body. A follow-up request for a deleted task should return HTTP 404; this is an additional check.

Running the project

Install dependencies with npm.cmd ci. Prepare a PostgreSQL database and configure DATABASE_URL in .env using .env.example. Run npm.cmd run db:setup, then npm.cmd run dev. PORT defaults to 3000, and FRONTEND_URL configures an additional allowed browser origin. The README provides full setup instructions and Postman examples.

Repository: github.com/kkovaycin/taskflow-api